

Preserving Requirements-Driven Development in the Age of AI: an Empirical Evaluation of the Impact of Software Specifications in LLM-Assisted Coding

Nicholas Caporusso
caporusson1@nku.edu

Jeffrey Perdue
perduej7@mymail.nku.edu

Northern Kentucky University
Highland Heights, KY USA

Abstract

This paper presents an empirical investigation into the role of requirements-driven development in Large Language Model (LLM)-assisted software engineering. As both professional developers and non-expert users increasingly rely on LLMs to generate code, growing concerns have emerged regarding software quality, transparency, and alignment with established engineering standards. While most LLMs can rapidly translate natural language prompts into executable programs, the lack of explicit requirements often results in brittle, undocumented, or incomplete implementations. To assess whether incorporating requirements as an intermediate reasoning step improves output quality, we conducted a comparative evaluation involving seven widely used LLMs. Each model was assigned coding tasks of varying complexity and prompted using two contrasting strategies: (1) direct prompt-based generation and (2) a structured, requirements-first approach in which models were explicitly instructed to produce Software Requirements Specifications (SRS) prior to implementation. Using ISO/IEC 25010 quality metrics, we assessed both functional and non-functional dimensions of the generated artifacts. Our findings demonstrate that integrating explicit requirements significantly enhances code robustness, maintainability, usability, and explainability across all models tested, regardless of architecture or training paradigm. In our study, LLMs treated the generated SRS as an execution checklist, adhering more strictly to specified constraints and reducing hallucinations or arbitrary design assumptions. These results suggest that requirements-first prompting can serve as an effective mechanism for embedding software engineering discipline into LLM workflows. We therefore advocate for treating SRS generation as a standard intermediate step within AI-assisted development pipelines.

Keywords: Large Language Models, Requirements Engineering, Software Requirements Specifications, Software Development Lifecycle

Preserving Requirements-Driven Development in the Age of AI: an Empirical Evaluation of the Impact of Software Specifications in LLM-Assisted Coding

Nicholas Caporusso and Jeffrey Perdue

1. INTRODUCTION

In recent years, the performance of Large Language Models (LLMs) has improved consistently across all tasks, including coding, where models can outperform humans in many programming challenges (Huang et al., 2025). The significant advances in models' ability to write code are transforming how engineers approach software development. LLM-based coding, encompassing tasks from code generation, completion, and refactoring to debugging and test case generation, promises to significantly augment developer productivity (Chen et al., 2021, Du et al., 2024). The potential benefits of LLM-aided coding are substantial, including accelerated development cycles, reduced manual effort, and improved code quality through automated suggestions and testing.

As a result, software engineers are increasingly using AI to write code and companies are integrating LLMs across all stages of the software development lifecycle (SDLC). In addition to professionals, the general audience, including people with non-technical backgrounds, is progressively using generative AI models to "vibe-code" their applications, facilitated by no-code platforms and by the integration of artifacts and runtime environments, which lower the barrier to entry for a variety of audiences to create software. Most popular LLMs enable users to translate natural language into functional code in one simple step. Furthermore, several platforms provide an environment where coding artifacts can be executed and shared with others.

Simultaneously, this novel approach to programming and its rapid adoption introduces a novel set of downsides and challenge established software engineering (SE) practices. Critical issues include the potential for generating erroneous or code due to the non-deterministic behavior of LLM outputs, traceability difficulties caused by the black-box nature of generative models, explainability concerns related to the lack of transparent reasoning processes in code generation, risks of hallucinations that compromise code quality, security vulnerabilities and fictitious package dependencies that open

doors to malicious intents, and interoperability issues stemming from inconsistent AI outputs across different contexts (Alenezi and Akour, 2025).

Focusing on the performance aspects of LLM-based programming is essential to address issues related to the quality of the output, including bugs, references to deprecated libraries, and runtime errors. To this end, current research focuses on developing robust validation and verification processes to ensure accuracy and reliability of AI-generated code (Siddique, 2024). Also, critical work involves implementing explainable AI tools to provide transparency in code generation (Zhong and Goel, 2024) and establishing governance frameworks for managing the long-term implication of AI-based coding. Nonetheless, more in-depth, theoretical reflections are needed on how to integrate software engineering processes and practices into LLMs. Vice versa, software engineering approaches require extensive modification when AI models are involved. For instance, empirical research studies found that traditional software requirements are too abstract for direct input into LLMs and must first be manually decomposed into smaller programming tasks (Ullrich et al., 2025).

This paper investigates the importance of integrating SE processes into LLM-based coding, and it presents the results of a study that analyzed the differences in the output generated by L

intermediate step similar to Chain of Thought (CoT), particularly in coding tasks that involve tackling problems that go beyond code snippet or simple challenges.

2. RELATED WORK

Most research focusing on LLM-based coding primarily specialized in benchmarking improvements in LLMs' ability to write code. For instance, the authors of (Huang et al., 2025) introduced EffiBench to evaluate AI systems across a diverse set of coding problems and programming challenges. Then, they used the system to compare the performance of 42 models. Coding ability benchmarks can be useful to rank LLMs and identify the most suitable models, though the speed at which LLMs are updated involves continuously revisiting the leaderboards.

LLMs are usually evaluated based on their ability in producing quality output, whether in coding or other content generation tasks. Nonetheless, the intersection of LLMs and software engineering has also generated substantial academic interest in the past few years. Most studies focused on adoption patterns across different user populations, particularly developers and software companies, identifying new challenges and opportunities to train and engage software developers in LLM use (e.g., finding new solutions to improve quality assurance) and analyzing the impact of AI along the entire SLDC (Nahar et al., 2024). For instance, studies exploring the integration of LLM and SE from requirements engineering to deployment proposed three distinct scenarios for how LLMs might be integrated into software engineering processes, each representing increasing levels of transformation: routine development tool integration, expert role replacement, and comprehensive process transformation (Belzner et al., 2023, Banh et al., 2025).

Recent literature surveys focusing on LLM-based code generation analyzed various dimensions of software engineering, from areas of applications to evaluation methodologies, highlighting the need of integrating SE approaches within LLMs tasks and architectures (Vasudevan and Reddivari, 2025). Furthermore, several works highlighted the lack of studies on the quality of the generated outputs when artifacts involve tasks that go beyond the programming challenges proposed in benchmarks, which usually have a limited scope (Wang and Chen, 2023, Fan et al., 2023). The authors of (Gao et al., 2025) presented a comprehensive analysis of

software engineering issues surfacing in the LLM era, and they identified 26 key challenges across seven aspects of SE, including requirements, coding assistance, testing, and maintenance. For instance, as documented in other works, their work emphasized that requirements as typically documented are too abstract for direct LLM input. As a result, they necessitate human intervention to break them down into smaller and more manageable programming tasks to translate them into effective prompts (Ullrich et al., 2025), making traditional approaches to writing SE requirements unfit for LLM contexts. Ahmad et al. (Ahmad et al., 2023) conducted a systematic mapping study on requirements engineering for AI systems, analyzing 43 primary studies. In addition to discussing how AI systems usually lack precise requirements and proper requirements engineering techniques, the authors also found that traditional SE practices need substantial adjustment in order to be utilized in the context of AI-enabled systems, in accordance with (Amershi et al., 2019).

At the opposite end of the development workflow, the inherent lack of explainability of the output of LLMs is among the most reported concerns of AI coding (Ma et al., 2024). Although AI can be utilized

Most popular LLMs directly output code from natural language prompts, skipping necessary steps including requirements analysis or making assumptions about missing information such as the implementation language. Although a few studies explored how Chain of Thought (CoT) could incorporate best practices to improve the SE performance of LLMs (Yang et al., 2024), current CoT methods provide insight into the model's reasoning rather than approaching coding tasks using SE standards. Requirements engineering has received less attention, and most studies focused on how LLMs can be utilized to generate software requirements from text rather than on integrating requirements engineering into the generation process. The authors of (Hymel and Johnson, 2025) have shown that LLM-generated specifications can be more aligned and complete than human-generated ones, performing at 720x the speed while costing only 0.06% of human expert time. However, researchers also found that traditional requirements artifacts must be significantly adapted for effective LLM utilization. This suggests the need for new approaches for integrating software specifications into LLM-based workflows. Sporadic studies such as (Sami et al., 2024), which showcased a demonstrator of a web interface that uses ChatGPT to produce requirements, do not address the core issue of how output generation can be improved by incorporating SE best practices in the process. In (Oswal et al., 2024), the authors showed that GPT-3.5 can translate free-text feature requests into structured user stories that satisfy four of the five attributes of the ISO/IEC/IEEE 29148-2018 standard on first pass. Follow-up research reported a 32% reduction in requirements modification requests when engineers were provided with an LLM-generated, schema-validated template versus a blank form (Korrapolu et al., 2025). These studies showed that LLMs still struggle with precision and highlight how vague or incomplete prompts can mislead both humans and models. In their experiments, simple clarifying questions led to more accurate requirements and fewer hallucinations. Although this is a promising direction, it also points to the need for iterative, human-in-the-loop workflows rather than one-shot generation.

A few recent frameworks take one step further in adopting SE practices by embedding requirements negotiation directly into the system. For instance, systems such as AutoGen and ChatDev (Qian et al., 2024), split tasks across multiple agents with defined roles (planner, coder, tester, for example) and use

structured dialogue to coordinate their work. These role-based pipelines simulate aspects of real-world engineering teams, where different stakeholders contribute to the spec before implementation begins. Researchers have found that this multi-agent model tends to produce more coherent outcomes, particularly when agents share a consistent set of expectations (Qian et al., 2024), and showed significant improvements in code generation, from accuracy to reliability, and improved runtime debugging (Ashrafi et al., 2025). Nevertheless, further research is needed to analyze the challenges and opportunities at the intersection between LLM-based code generation and SE practices.

3. STUDY

The ultimate goal of our work is to gain insight into how LLM-assisted development can be integrated with established software engineering methodologies, particularly, requirements-driven development, to improve overall project management, including code generation, testing, and maintenance. The objective of this study is to empirically evaluate and compare the adherence of different models to SE best practices when they are prompted to generating code. Specifically, we were interested in examining whether popular models naturally incorporate requirement analysis or collect specifications from users when given minimal prompts, and how their behavior and the quality of their output change when they are explicitly asked to (1) generate software

Several repositories and benchmarks offer software development tasks for evaluating the ability of LLMs to write code (e.g., HumanEval and MBPP) (Yu et al., 2024). However, regardless of their intrinsic difficulty, these tasks often consist in writing small snippets, designing technical-interview type algorithms, and solving programming challenges designed to compare LLMs in terms of their coding proficiency. Additionally, they often overrepresent domains familiar to the researchers (e.g., data structures, LeetCode-style puzzles), rather than evaluating a model's general adherence to best practices in software design and development. On the contrary, our goal was to test LLMs in their ability to code a complete application, which involves a different type and scale of reasoning (i.e., systemic thinking) and complexity and requires a systematic approach typical of SE skills.

Therefore, we asked each of the LLMs to generate a list of nine projects (i.e., three for each difficulty level). Also, by asking each LLM to suggest candidates, we aimed to anchor the task space to what that model believes is tractable with its internal knowledge and token budget, which, in turn, reduces the risk of giving any system an assignment that is trivial for one architecture but impossible for another, improving fairness across models. The following prompt was utilized. "I am designing a study to evaluate an LLM's ability to write software requirements and code. Define nine software projects that can be developed by an LLM. Three projects must be easy, three must have medium difficulty, and three must be more challenging to implement. For each project, write a title and short implementation description (not the full requirements)." The prompt specifically asked LLMs to maintain consistency across LLMs. As a result, each model generated nine project descriptions, resulting in a pool of candidates where several proposals were similar across different LLMs.

Subsequently, we finalized a shortlist of three projects (one per difficulty level) using a consensus system in which we selected the applications proposed by the most models. The resulting project descriptions (listed below) were revised to enhance clarity and consistency, ensuring they were representative, feasible, and appropriately scoped for empirical evaluation.

- *Low difficulty.* To-do list application: a software where users can add, delete, and mark tasks as complete. Proposed by six models (86% of models).
- *Medium difficulty.* Basic chatbot: a software where a conversational agent responds to predefined frequently asked questions about

a specific topic. Proposed by five models (71% of models).

- *High difficulty.* Real-time collaborative text editor: a software where multiple users can edit the same document simultaneously. Proposed by five models (71% of models).

Experimental tasks

For each project, all the models were prompted with the experimental tasks described below, which were executed in separate conversations.

- Task I - Default output. In this task, models were asked to develop each application with minimal instruction. Specifically, the prompt utilized is as follows. "Implement the following <APPLICATION DESCRIPTION>". This condition establishes a baseline measure of how LLMs approach coding tasks by default, without explicit guidance concerning

is for users to understand, learn, operate, and interact with the software.

- Reliability, i.e., the software's ability to maintain performance, recover from failures, and handle errors over time.
- Security aspects, such as potential risks due to insecure code and protection of data and systems through confidentiality, integrity, authentication, and access control.
- Maintainability, i.e., how easily the software can be modified, understood, tested, reused, or analyzed for defects.
- Portability, that is, the ease with which the software can be transferred to and installed on different platforms or environments.

Text artifacts, including SRS, usage guidelines, CoT trails, are evaluated separately. Specifically, the SRSs generated by LLMs in Task II are evaluated according to the following criteria, based mainly on the IEEE 830 and ISO/IEC/IEEE 29148 standards.

- *Product perspective*, describing the application and explaining how the system fits into a larger context (system-of-systems, legacy replacement, etc.).
- *Product functions/Functional requirements* listing major capabilities the system will provide in business language.
- *User classes and characteristics* describing user roles, skills, and needs that shape requirements and UI design.
- *Operating environment*, which comprises hardware, OS, network, or platform constraints.
- *Design and implementation constraints*, which capture mandated technologies, standards, legal constraints that limit design freedom.
- *Assumptions and dependencies* stating external factors the project relies on (e.g., third-party APIs, hardware availability).

Some aspects of the generated artifacts (e.g., portability) are analyzed qualitatively, rather than quantitatively, to address the inherent complexity of the diverse types of output.

4. RESULTS

Our analysis was two-fold. First, we compared the code artifacts of the two tasks. To this end, we mapped observable traits to each ISO/IEC 25010 characteristic, assigning ratings on a 5-point Likert scale per model and dimension. These results are summarized in Table 1. Subsequently, we focused on the requirements generated by the models to normalize the performance of each LLM in Task II based on the quality of the

requirements.

Attribute	Task I Avg	Task II Avg	Δ (T2 - T1)
FS	3.50	4.25	0.75
PE	3.12	4.00	0.88
CO	3.38	4.00	0.62
US	3.50	4.00	0.50
RE	3.00		

Grok produced a responsive client-side web application in HTML, JavaScript, Tailwind CSS. Its response referenced the technologies used and mentioned the reason for its implementation choices as “no specific language was mentioned, so I’ll bias towards a web technology solution”. In addition to a minimalistic code artifact, the output also included a quick user guide.

Perplexity yielded the best implementation, providing separate artifacts for code, documentation, references, execution steps, and a user guide. Additionally, it included executable code artifacts, easily downloadable source code, and an in-depth guide to the application, along with initial technical documentation.

Qwen created an elementary command line interface (CLI) application developed in Python. The text accompanying the code artifact included minimal documentation on usage. All the applications worked (ChatGPT’s implementation executed correctly on a local computer). All the models made assumptions regarding the implementation technologies. Although the lack of implementation aspects (e.g., programming language, deployment environment) could be a deliberate choice to lower the access barrier, it renders the output less traceable.

Task II – SRS guided development

As shown in Figure 1, **Claude**’s SRS-driven implementation significantly expanded the functional suitability, adding priority selection for tasks, undo functionality for deletions, sorting options, and real-time search functionality. Input validation was also included. From a usability standpoint, the interface was further refined with the addition of priority badges, live counters, hover effects, and keyboard shortcuts. Also, the application included keyboard navigation and semantic HTML for enhanced accessibility. Claude’s Task II implementation was a substantially more comprehensive and robust application. It not only added many advanced functional features (search, sort, priority, undo) but also explicitly addressed several non-functional quality attributes, including reliability (error handling, undo), performance, maintainability, and accessibility.

Deepseek’s Task II implementation represented a fundamental shift in technology and scope. Moving from a limited desktop GUI to a robust, feature-rich web application with data persistence directly addressed the limitations of the Task I output. Specifically, the code artifact consisted of a web-based React.js application that incorporated significant new features, including

descriptions, priorities, due dates, editing existing tasks, and bulk actions (i.e., completing or deleting multiple tasks). Filtering by status and sorting by due date/priority were also added. In Task II, DeepSeek also included data persistence between browser sessions (i.e., localStorage). From a portability standpoint, the shift to React vastly improved the output compared to the Tkinter desktop app from Task I.

Gemini’s implementation consisted of a minimal code snippet in Python representing a minor enhancement compared to Task I. Although the text accompanying the code stated that the solution provided a complete, standalone Python application per the requirements outlined in the SRS, the code only included partial features.

GPT 4o added optional task descriptions and data persistence using localStorage, directly addressing a missing feature from Task I. GPT o3’s Task II implementation

reliability, directly addressing the core limitations and suggested enhancements identified in its Task I output.

Model	PP	FR	UC	OE	DC	AD	Score
Claude	1	1	1	1	1	1	6
GPT-3.5	1	1	1	1	1	0	5
GPT-4o	1	1	1	1	1	0	5
Deepseek	0	1	1	1	1	1	5
Gemini	0	1	1	1	0	1	4

Table 2: SRS evaluation scores

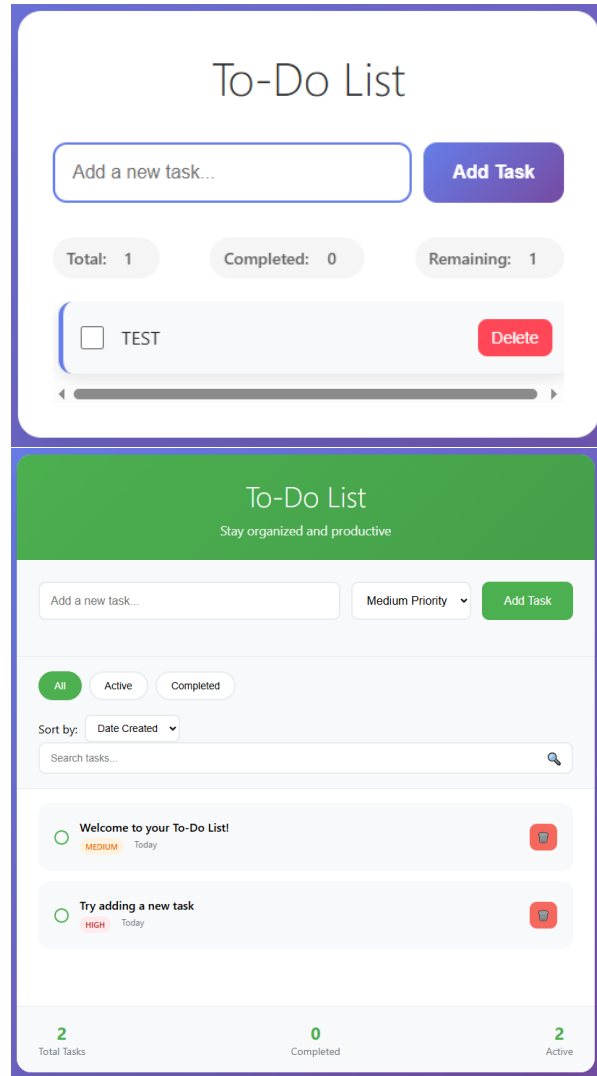


Fig 1. GUI of the low-difficulty application implemented in Task I (top) and Task II (bottom).

5. DISCUSSION

SRS-driven development leads to better code artifacts

Adopting a requirements-first prompting strategy

consistently improved quality across all ISO 25010 dimensions, even without any changes to the generated SRSs. Implementations gained persistence and richer user experiences. All the LLMs implemented the to-do list project as a web application, except Gemini, which continued to utilize Python. This might have

detailed specifications do not result in a higher number of lines of code, but also in substantially feature-rich applications. Although GPT-o3's generated code repository is larger, Claude's code artifact shows that fine-grained user interface requirements produce depth (e.g., accessibility, undo, stats) rather than breadth. 5.0.4 Project scope might influence results Although our study provides strong evidence for the benefits of requirements-driven development in LLM-assisted coding, this paper considers a simple project only to allow for a controlled comparison. Therefore, our findings might not fully generalize to more complex, large-scale software projects that involve intricate architecture, diverse programming paradigms, and extensive team collaboration.

Human-in-the-loop is essential

Our study primarily focused on a single-turn interaction for both default and requirements-driven tasks. While iterative, human-in-the-loop workflows are crucial for refining requirements and improving output quality, our experimental design aimed to establish a baseline comparison between two distinct prompting strategies. In future studies, we will investigate the impact of iterative refinement on the quality of LLM-generated code and specifications. This would provide a more realistic representation of real-world software development scenarios.

6. CONCLUSION AND FUTURE WORK

In this paper, we described a study aimed to assess whether popular LLMs naturally incorporate requirement analysis or collect specifications from users when given minimal prompts, and how their behavior and output quality change when explicitly asked to (1) generate software requirements specifications and (2) produce code based on those generated specifications. We evaluated the LLMs based on their performance in developing a to-do list application, which is a low-difficulty yet comprehensive project. Task I represented the default, minimal-instruction approach, serving as a baseline, while Task II involved a structured, requirements-first approach.

A comparative evaluation of the implementations provided by the selected LLMs was realized using quality metrics based on the ISO/IEC 25010 standard. Our findings strongly support the hypothesis that incorporating explicit software requirements as an intermediate step between basic prompt and code generation significantly enhances output quality across both functional

and non-functional dimensions. Models guided by formal specifications consistently produced more robust, maintainable, and feature-rich artifacts, often directly addressing limitations or possible enhancements identified in their default outputs.

Although our evaluation focused on a low-difficulty project to ensure tractability across all selected LLMs, we will extend this empirical evaluation to medium- and high-difficulty projects in our future work, as identified during our project generation and selection phase, to assess the scalability and robustness of the requirements-first approach.

7. REFERENCES

- Ahmad, K., Abdelrazek, M., Arora, C., Bano, M., and Grundy, J. (2023). Requirements engineering for artificial intelligence systems: A systematic mapping study. *Information and Software Technology*, 158:107176. DOI:

- <https://doi.org/10.1016/j.infsof.2025.107751>
- Belzner, L., Gabor, T., and Wirsing, M. (2023). Large Language Model Assisted Software Engineering: Prospects, Challenges, and a Case Study, page 355–374. Springer Nature Switzerland. DOI: https://doi.org/10.1007/978-3-031-46002-9_23
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. (2021). Evaluating large language models trained on code. ArXiv ID: <https://arxiv.org/abs/2107.03374>
- Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X., and Lou, Y. (2024). Evaluating large language models in class-level code generation. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, page 1–13. ACM. DOI: <https://doi.org/10.1145/3597503.3639219>
- Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., and Zhang, J. M. (2023). Large language models for software engineering: Survey and open problems. In 2023 IEEE/ACM International Conference on Software Engineering (ICSE-FoSE), pages 31–53. DOI: <https://doi.org/10.1109/icse-fose59343.2023.00008>
- Gao, C., Hu, X., Gao, S., Xia, X., and Jin, Z. (2025). The current challenges of software engineering in the era of large language models. ACM Transactions on Software Engineering and Methodology, 34(5):1–30. DOI: <https://doi.org/10.1145/3712005>

- Journal. DOI:
<https://doi.org/10.2139/ssrn.4885910>
- Ullrich, J., Koch, M., and Vogelsang, A. (2025). From requirements to code: Understanding developer practices in llm-assisted software engineering. ArXiv ID:
<https://arxiv.org/abs/2507.07548>
- Vasudevan, P. and Reddivari, S. (2025). The role of generative ai models in requirements engineering: A systematic literature review. In Proceedings of the 2025 ACM Southeast Conference, pages 188–194. DOI:
<https://doi.org/10.1145/3696673.3723053>
- Wagner, S., Barón, M. M., Falessi, D., and Baltes, S. (2025). Towards evaluation guidelines for empirical studies involving llms. DOI:
<https://doi.org/10.1109/wsese66602.2025.00011>
- Wang, J. and Chen, Y. (2023). A review on code generation with llms: Application and evaluation. In 2023 IEEE International Conference on Medical Artificial Intelligence (MedAI), page 284–289. IEEE. DOI:
<https://doi.org/10.1109/medai59581.2023.00044>
- Yang, G., Zhou, Y., Chen, X., Zhang, X., Zhuo, T. Y., and Chen, T. (2024). Chain-of-thought in neural code generation: From and for lightweight language models. IEEE Transactions on Software Engineering, 50(9):2437–2457. DOI:
<https://doi.org/10.1109/tse.2024.3440503>
- Yu, Z., Zhao, Y., Cohan, A., and Zhang, X.-P. (2024). HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation Task. DOI:
<https://doi.org/10.18653/v1/2025.findings-acl.686>
- Zhong, C. and Goel, S. (2024). Transparent ai in auditing through explainable ai. Current Issues in Auditing, 18(2):A1–A14. DOI:
<https://doi.org/10.2308/ciia-2023-009>